

MetrCheck AI — System Connectivity & Integration Guide

Comprehensive Architectural Breakdown of Client-Server, Database, AI Pipeline, and Statutory Integrations

EXECUTIVE SUMMARY & INTEGRATION TOPOLOGY:

MetrCheck AI implements a modular, high-throughput, and fault-tolerant architecture designed to automate statutory label compliance under the **Legal Metrology (Packaged Commodities) Rules, 2011** and **FSSAI Regulations, 2020**. All system connections are decoupled into 5 operational tiers: **(1) Client-to-Backend REST Gateway** with development & production reverse proxying; **(2) Asynchronous Relational Persistence** using SQLite in WAL mode; **(3) 7-Stage Internal Pipeline Orchestration** linking Computer Vision, PaddleOCR PP-OCRv4, Contextual Regex Parsing, and Codified Legal Metrology Rules; **(4) External Statutory & Infrastructure Verifications** (FoSCoS FSSAI API, GS1 DataKart GTIN Barcode API, OpenCV ArUco Physical Metric Calibration, and SMTP Mail Delivery); and **(5) Multi-Device & Field Connectivity** (LAN IP auto-detection with Terminal ASCII QR code, Cloudflare HTTPS zero-prompt tunneling, and native Capacitor Android APK compilation).

1. Frontend-to-Backend Client-Server Connection

Communication between the **React 19 / Vite** frontend and the **FastAPI** Python backend is handled through asynchronous HTTP REST endpoints over JSON and multipart form data. Multiple reverse proxying and CORS safeguards ensure seamless local, network, and containerized operation.

Mechanism	Implementation / File	Technical Details & Operational Functionality
Vite Dev Reverse Proxy	<code>frontend/vite.config.ts</code>	Routes all requests starting with <code>/api</code> and <code>/uploads</code> to <code>http://127.0.0.1:8000</code> . Completely eliminates browser Cross-Origin Resource Sharing (CORS) friction during development.
Production Nginx Proxy	<code>frontend/nginx.conf</code> <code>docker-compose.yml</code>	In Docker deployment, Nginx listens on port 80/8080 and reverse-proxies <code>/api</code> and <code>/uploads</code> to the internal Docker service container <code>http://backend:8000</code> while preserving client IP headers.
Centralized API Client	<code>frontend/src/services/api.ts</code>	Wraps browser <code>fetch()</code> with dynamic host detection (reads <code>localStorage: metrcheck_api_url</code> or <code>VITE_API_URL</code>), unified error handling, and automated 401 session expiration handling.
Bearer Token Interceptor	<code>frontend/src/services/api.ts:authHeaders()</code>	Pulls stored authentication token from <code>localStorage (metrcheck-token)</code> and injects <code>Authorization: Bearer <token></code> into every protected HTTP request header.
FastAPI CORS Middleware	<code>backend/main.py</code> <code>backend/config.py</code>	Enables <code>CORSMiddleware</code> with configurable <code>CORS_ORIGINS</code> (default <code>['*']</code>), allowing direct cross-origin API calls from mobile browsers, local LAN IPs, or tunneled hostnames.
Multipart Multi-Panel Ingestion	<code>POST /api/analyze</code> <code>api.ts:analyzeProducts()</code>	Packages 1 to 4 image files (Front, Back, Sides) alongside JSON panel metadata in a single <code>FormData</code> payload with automated MIME validation.

2. Backend-to-Database Connection (Persistence Architecture)

Persistence is achieved through a high-concurrency asynchronous SQLite connection managed via `aiosqlite` in `backend/database/db.py`.

Key Database Connection Features:

- **Write-Ahead Logging (WAL Mode):** Initialized on every connection via `PRAGMA journal_mode=WAL;`. Enables concurrent readers while writes occur without database locking.
- **Busy Timeout Guard:** Configured with `PRAGMA busy_timeout=5000;` to ensure transactions retry for up to 5 seconds under heavy multi-panel inspection bursts before raising lock exceptions.
- **FastAPI Lifespan Startup:** `init_db()` runs automatically at application boot, creating or upgrading tables idempotently (analyses, users, password_resets, account_audit_logs).
- **Automated Test Isolation Guard:** `_check_safety_guard()` inspects `TEST_MODE` and `sys.modules['pytest']`. If automated tests attempt to connect to the production `metrc_check.db`, execution is immediately aborted to prevent data corruption.

3. Internal Pipeline & Multi-Service Orchestration

When an inspection request arrives, `backend/services/analysis_service.py:analyze_products()` coordinates an asynchronous 7-stage processing pipeline connecting internal computer vision, deep learning, statutory parsing, and scoring engines.

Stage	Internal Component	Data Ingested & Operational Transition
1. Image Ingestion	<code>services/image_service.py</code> <code>utils/validators.py</code>	MIME whitelist check (PNG, JPEG, WebP) and 10MB file limit. Generates UUID and writes sanitized files to <code>backend/uploads/</code> .
2. CV Preprocessing	<code>ocr/preprocessing.py</code> <code>ocr/quality.py</code>	Performs EXIF rotation correction, 4-point homography perspective warp, Lanczos upscaling, and Laplacian variance blur & contrast quality evaluation.
3. Deep Learning OCR	<code>ocr/factory.py</code> <code>ocr/paddle_engine.py</code>	Singleton PaddleOCR PP-OCRv4 (DBNet text detection + SVTR recognition) runs concurrently across panels with thread-safe locks and Windows OneDNN/PIR workarounds. Returns words and 2D bounding boxes.
4. Text Normalization	<code>ocr/cleaner.py</code> <code>ocr/repair.py</code>	Strips OCR artifacts while preserving currency (₹), units, and dates. Applies regex repair heuristics for swapped letters (e.g. FSSAI digits, Net Qty '9' → 'g').
5. Structured Extraction	<code>extraction/extractor.py</code> <code>extraction/patterns.py</code>	Contextual regex parser extracts MRP, Net Quantity, Dates, Manufacturer Name/Address/PIN, Consumer Helpline, Barcodes, and Country of Origin into a typed <code>ProductInfo</code> schema.
6. Statutory Rules	<code>compliance/engine.py</code> <code>compliance/rules/</code>	Validates declarations against 14 codified rules (LM-001..LM-009 & FS-001..FS-005). <code>scorer.py</code> computes weighted points (0–100 score).
7. Dossier Generation	<code>services/report_service.py</code> <code>enforcement/penalties.py</code>	Calculates Legal Metrology Act Sec 36/38 penalty brackets, produces court-ready show-cause notices, generates PDF/XLSX/CSV/JSON reports, and persists record to SQLite.

4. External Services & Statutory Integrations

MetrCheck AI connects to external regulatory databases and physical measurement standards with resilient offline/cache fallbacks:

Integration Target	Protocol / Location	Operational Function & Fallback Resilience
FSSAI FoSCoS Verification	REST API (HTTPS) <code>integrations/fssai/verifier.py</code>	Connects to government FoSCoS API to verify 14-digit licence validity and business name. Fallback: Validates mathematical 14-digit structure (starting with 1 or 2), parses state jurisdiction code, and queries local offline cache.
GS1 DataKart GTIN Barcode	REST API (HTTPS) <code>integrations/gs1/verifier.py</code>	Connects to GS1 DataKart cloud to verify product GTIN barcode ownership and registered brand name. Fallback: Executes standard GS1 Modulo-10 check-digit algorithm and checks local GTIN cache.
Physical ArUco Calibration	OpenCV Computer Vision <code>services/calibration_service.py</code>	Detects physical 50mm ArUco fiducial calibration markers placed on packaging artwork to calculate real-world pixel-to-millimeter ratio. Evaluates statutory Rule 12 minimum font heights. Fallback: DPI factor (0.18 mm/px).
SMTP Email Server	SMTP / STARTTLS (Port 587) <code>auth/security.py</code>	Connects to enterprise mail server (e.g. Gmail/Gov SMTP) to deliver officer invitation tokens and secure password reset links. Fallback: Generates in-app dev activation tokens when SMTP is unconfigured.

5. Cross-Device, Mobile & Field Network Connectivity

MetrCheck AI provides three dedicated pathways to connect field inspection devices, smartphones, and tablets to the processing server:

Connectivity Mode	Configuration Script	Architecture & End-User Experience
Local Wi-Fi Network & ASCII QR Code	<code>runner.py</code> <code>allow_firewall.bat</code>	<code>runner.py</code> probes active LAN IP via UDP socket (8.8.8.8) and prints an interactive ASCII QR Code in the terminal. Inspectors scan the QR code with their mobile camera to connect instantly via <code>http://<LAN_IP>:5173</code> . Firewall batch script ensures ports 5173 and 8000 are unblocked.
Cloudflare Zero-Prompt Tunnel	<code>tunnel.bat</code>	Spawns a public HTTPS cloud tunnel via <code>npx cloudflared tunnel --url http://localhost:5173</code> . Generates a secure, temporary public URL (e.g. <code>https://xyz.trycloudflare.com</code>) that allows judges, remote officers, or phones to test without being on the same local Wi-Fi.
Native Android APK Bridge	<code>build_apk.bat</code> <code>frontend/android/</code>	Bundles React frontend via Vite, synchronizes web assets into Capacitor Android wrapper, and compiles native debug APK (<code>app-debug.apk</code>) with direct hardware camera and gallery capture permissions.

6. Authentication & Security Layer Wiring

MetrCheck AI secures all inter-service operations with zero third-party cryptographic dependencies, utilizing Python's built-in `hashlib` and `hmac` modules:

- **Password Hashing:** Salted with `os.urandom(16)` and hashed using **PBKDF2-HMAC-SHA256** with **200,000 iterations**. Passwords are never stored in plaintext, and comparisons use `hmac.compare_digest` to defeat timing attacks.
- **Stateless HMAC-SHA256 Bearer Tokens:** Encodes user identity, role, issue time, and expiration (480 minutes). Signed with server-side `SECRET_KEY`.
- **Role Hierarchy:** Enforced via FastAPI dependency injection (`require_roles(*roles)`):
 - **ADMIN:** Full system access, user provisioning, database purge, and audit logs.
 - **ENFORCEMENT_OFFICER:** Analysis, history, Sec 36/38 statutory penalty estimator, show-cause notices.
 - **MERCHANT_PUBLIC:** Multi-panel upload, listing text screening, and inspection report downloads.

7. Master Connectivity, Protocol & Port Matrix

Source	Destination	Port / Protocol	Key Module / Endpoint
Browser / Phone	Vite Dev Server	TCP 5173 (HTTP)	<code>frontend/vite.config.ts</code>
Vite Dev Server	FastAPI Backend	TCP 8000 (HTTP Proxy)	<code>vite.config.ts:proxy</code>
Nginx Container	FastAPI Backend	TCP 8000 (HTTP Proxy)	<code>frontend/nginx.conf</code>
FastAPI App	SQLite File	Async File I/O (WAL)	<code>backend/database/db.py</code>
FastAPI App	PaddleOCR Engine	In-Process PyPaddle	<code>backend/ocr/paddle_engine.py</code>
FastAPI App	FoSCoS API	TCP 443 (HTTPS)	<code>integrations/fssai/verifier.py</code>
FastAPI App	GS1 DataKart	TCP 443 (HTTPS)	<code>integrations/gs1/verifier.py</code>
FastAPI App	SMTP Server	TCP 587 (STARTTLS)	<code>backend/auth/security.py</code>
Mobile Device	Cloudflare Edge	TCP 443 (HTTPS Tunnel)	<code>tunnel.bat</code>